Course: 04733 AIOps **September 29, 2026**

Carnegie Mellon University Africa

Name: Martin Koome

Course: 04733 AIOps, Fall 2026

Lab 2

Monitoring Istio Metrics with Prometheus and Grafana

1. Overview

This lab revisits the previous week's fault-injection experiment on the Online Boutique application, this time using Prometheus queries and Grafana dashboards instead of Kiali. The focus is the `shippingservice`, which can be reached through two call paths: directly from the `frontend`, or from the `checkoutservice`. Istio's `istio_request_duration_milliseconds` histogram is used to measure the 99th percentile response time on each path, first under healthy conditions, then with a 1 s delay injected on the checkout path, and finally with the delay restricted to a single, custom-labelled `checkoutservice` pod.

1.1. Environment preparation

The updated deployment files (`deploy_yamls_v3.zip`) were unzipped into `./kustomize/base`. buckets match Istio's millisecond units. The manifests were then reapplied from the root of the Boutique directory, and the Prometheus dashboard was opened from the Istio folder:

```
kubectl apply -k kustomize/
istioctl dashboard prometheus
```

Listing 1: Setup: reapplying the v3 deployment files and opening Prometheus

2. Task 1 – Monitor Boutique with Prometheus

2.1. Query design

The query computes the 99% quantile of the Shipping service response time over a 5-minute `rate()` window, measured from the *source* side (`reporter="source"`) because the Istio delay is applied in the client-side proxy and is therefore invisible to the destination. A regular-expression matcher selects both callers, and `sum by (le, source_canonical_service)` keeps the histogram buckets (`le`) while preserving a separate series for each caller, so that a single query returns one p99 value per call path.

```
histogram_quantile(0.99, sum by (le, source_canonical_service)
  (rate(istio_request_duration_milliseconds_bucket{
     destination_canonical_service="shippingservice",
     source_canonical_service=~"checkoutservice|frontend",
     reporter="source"}[5m])))
```

Listing 2: Task 1: PromQL query for the graph view (p99 per caller)

For the table view, a range (subquery) suffix `[30m:1m]` was appended, which evaluates the same expression once per minute over the last 30 minutes, giving 30 values for each of the two callers.

```
histogram_quantile(0.99, sum by (le, source_canonical_service)
  (rate(istio_request_duration_milliseconds_bucket{
     destination_canonical_service="shippingservice",
     source_canonical_service=~"checkoutservice|frontend",
     reporter="source"}[5m])))[30m:1m]
```

Listing 3: Task 1: PromQL query for the table view with the `[30m:1m]` subquery

2.2. Results

Figure 1 shows the graph view over the past 30 minutes. Under healthy conditions the checkout path's p99 latency stays between roughly 3 and 4.7 ms, with a brief dip near 07:24, while the frontend path is flat at 2 ms. The flat frontend value most likely reflects the histogram bucket resolution: when nearly all observations fall into the same bucket, `histogram_quantile` returns the same interpolated value at every step. Figure 2 shows the corresponding per-minute table, split across two screenshots.

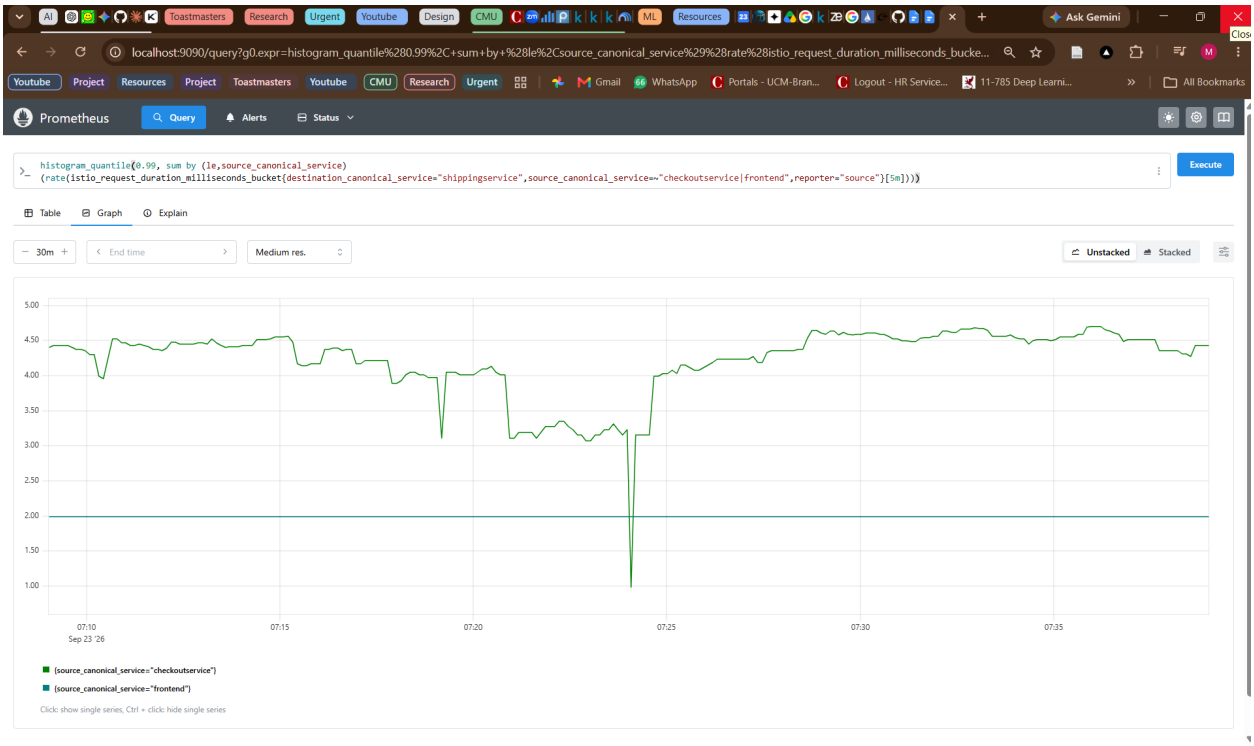
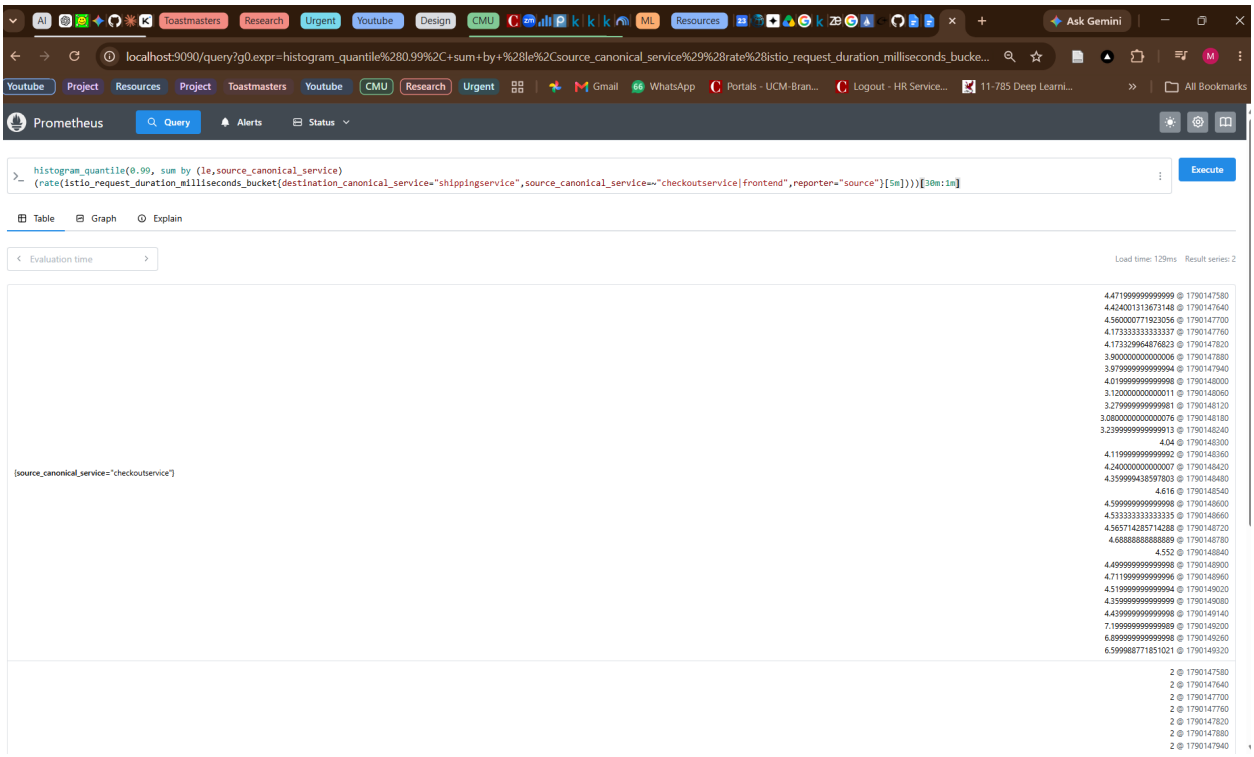
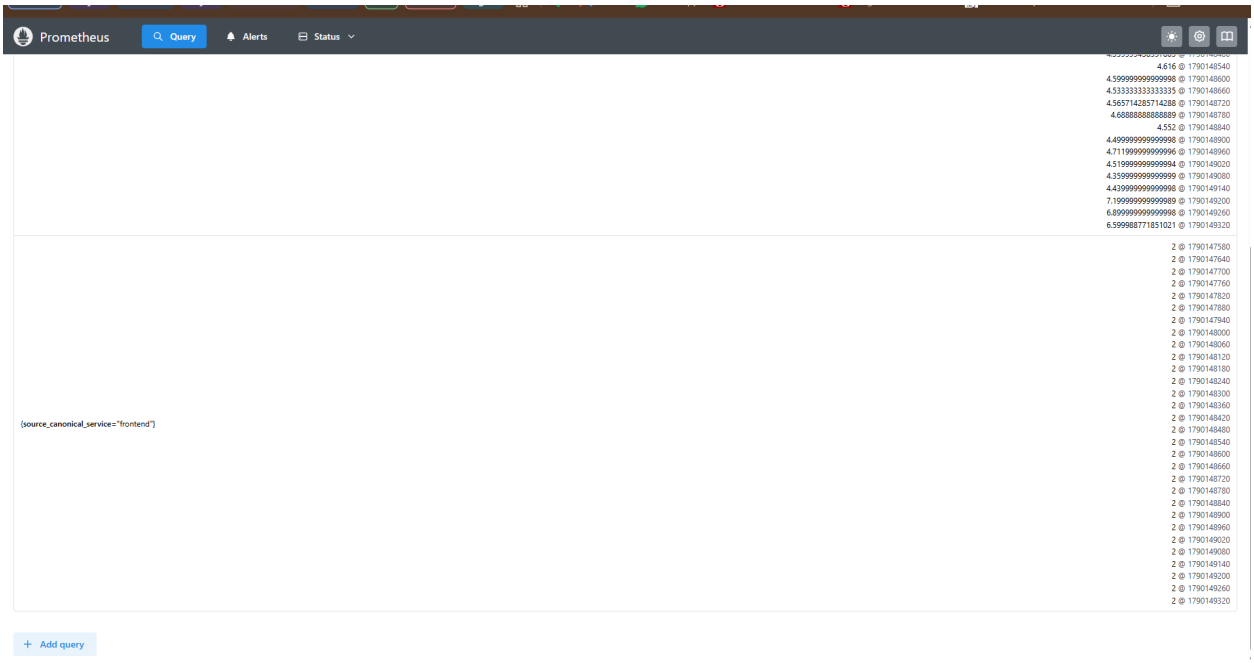


Figure 1: Task 1 – Prometheus graph view (last 30 minutes) of the p99 Shipping service response time, split by caller (`checkoutservice` and `frontend`).



(a) Task 1 – Table view (part 1): query and `checkoutservice` values.



(b) Task 1 – Table view (part 2): `frontend` values.

Figure 2: Task 1 – Prometheus table view using the `[30m:1m]` subquery: 30 per-minute p99 values for each of the two callers.

3. Task 2 – Selective Delay on the Checkout Path (Part 1)

3.1. Fault injection rule

A `VirtualService` for `shippingservice` was applied with an HTTP fault rule that matches only requests whose source carries the label `app: checkoutservice` and delays 100% of them by a fixed 1 s. A second, unconditional route handles all other traffic, so calls originating from the `frontend` are not affected.

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: shippingservice
spec:
  hosts:
  - shippingservice
  http:
  - match:
    - sourceLabels:
        app: "checkoutservice"
    fault:
      delay:
        fixedDelay: 1s
        percentage:
          value: 100
    route:
    - destination:
        host: shippingservice
  - route:
    - destination:
        host: shippingservice
```

Listing 4: Task 2: Istio fault-injection rule delaying calls from `checkoutservice` by 1 s

3.2. Results

The Task 1 queries were rerun unchanged. In Figure 3 the checkout series rises sharply at around 08:09, from about 4 ms to just under 1000 ms, once the rule takes effect, while the frontend series remains at 2 ms throughout. The per-minute table in Figure 4 confirms this: the checkout values climb from about 4.5 ms to a steady 995–998 ms, whereas every frontend value remains 2 ms. The rule therefore delays only the intended call path.

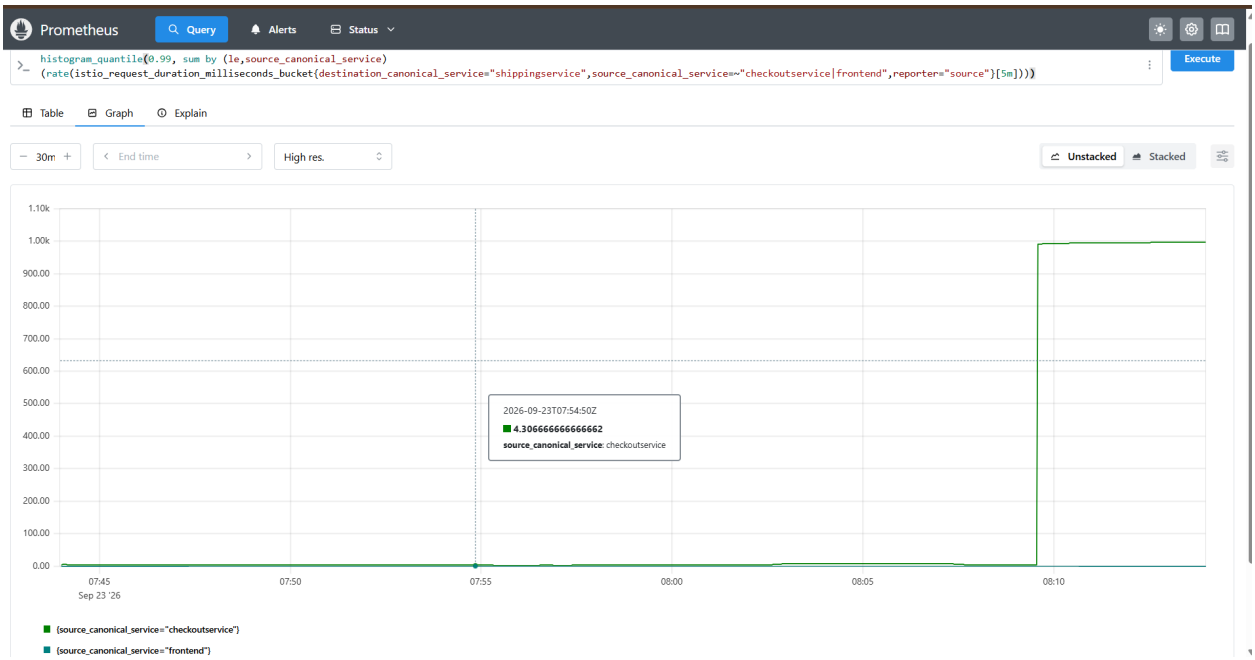
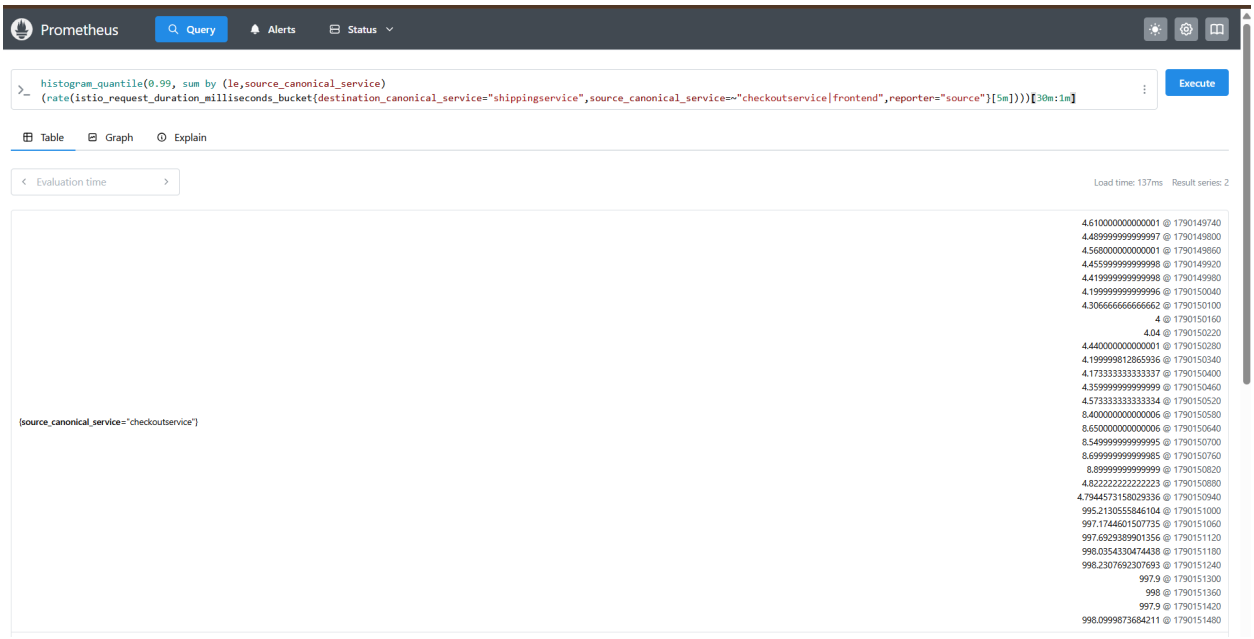
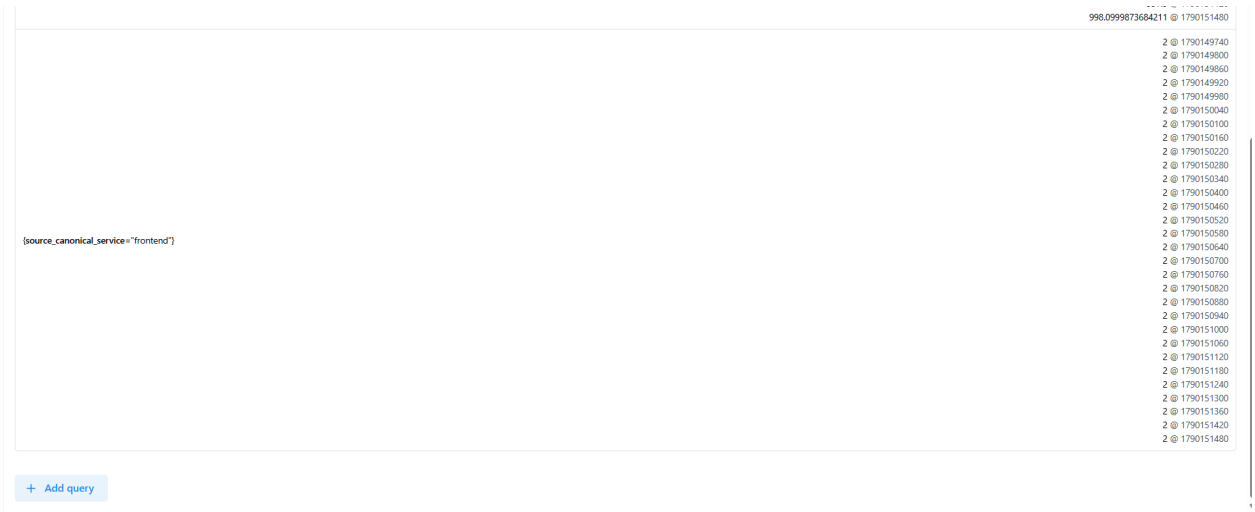


Figure 3: Task 2 – Prometheus graph view after injecting a 1 s delay on calls from `checkoutservice`: the checkout path jumps to about 1000 ms while the frontend path is unchanged.



(a) Task 2 – Table view (part 1): query and `checkoutservice` values rising to about 998 ms.



(b) Task 2 – Table view (part 2): `frontend` values unchanged at 2 ms.

Figure 4: Task 2 – Prometheus table view (`[30m:1m]`) after fault injection, showing the separate impact on the two call paths.

4. Task 3 – Selective Delay on One Checkout Pod (Part 2)

4.1. Scaling out and labelling one pod

The `checkoutservice` deployment was scaled to two replicas, and the custom label `fault=delay` was attached to one of the two pods only:

```
kubectl scale deployment checkoutservice --replicas=2
kubectl label pod checkoutservice-9ff5b8cbd-qhc9n fault=delay
kubectl get pods -l app=checkoutservice --show-labels
```

Listing 5: Task 3: scaling out `checkoutservice` and labelling one pod

The fault rule from Task 2 was then narrowed by adding the label `fault: "delay"` to its `sourceLabels` match, so that the 1 s delay applies only to requests coming from the labelled pod.

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: shippingservice
spec:
  hosts:
  - shippingservice
  http:
  - match:
    - sourceLabels:
        app: "checkoutservice"
        fault: "delay"
    fault:
      delay:
        fixedDelay: 1s
        percentage:
          value: 100
    route:
    - destination:
        host: shippingservice
  - route:
    - destination:
        host: shippingservice
```

Listing 6: Task 3: Istio fault-injection rule matching only the `fault=delay` checkout pod

Figure 5 shows the applied rule together with the pod listing. It confirms that only the pod `checkoutservice-9ff5b8cbd-qhc9n` carries the `fault=delay` label.

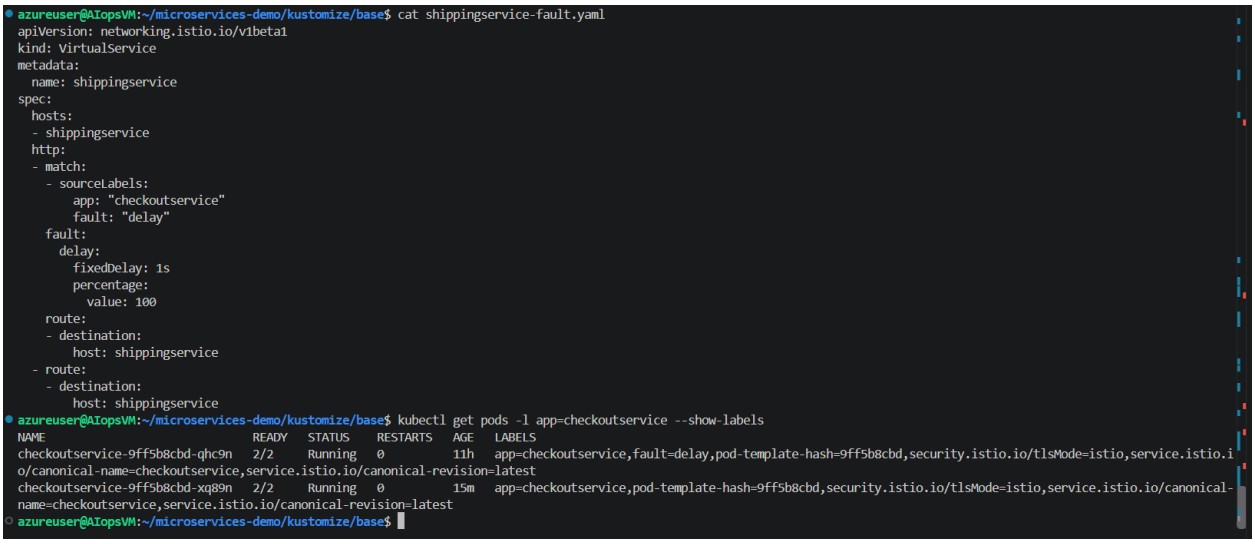


Figure 5: Task 3 – Applied fault-injection rule (`shippingservice-fault.yaml`) and the two `checkoutservice` pods; only `qhc9n` carries `fault=delay`.

4.2. Query design

This query considers only requests from `checkoutservice`. Because the metrics are reported from the source side, the `pod` label identifies the calling checkout pod, so grouping by `(le, pod)` produces one p99 series per pod within a single query.

```
histogram_quantile(0.99, sum by (le, pod)
  (rate(istio_request_duration_milliseconds_bucket{
     destination_canonical_service="shippingservice",
     source_canonical_service="checkoutservice",
     reporter="source"}[5m])))
```

Listing 7: Task 3: PromQL query separating the p99 latency of the two checkout pods

4.3. Results

In Figure 6 the labelled pod (`qhc9n`) stays constant at about 999 ms over the whole hour, while the unlabelled pod (`xq89n`) stays close to zero on the same scale. The table view in Figure 7 gives the current values: 999 ms for `qhc9n` and about 8.3 ms for `xq89n`. The delay is therefore confined to one of the two replicas of the same service.

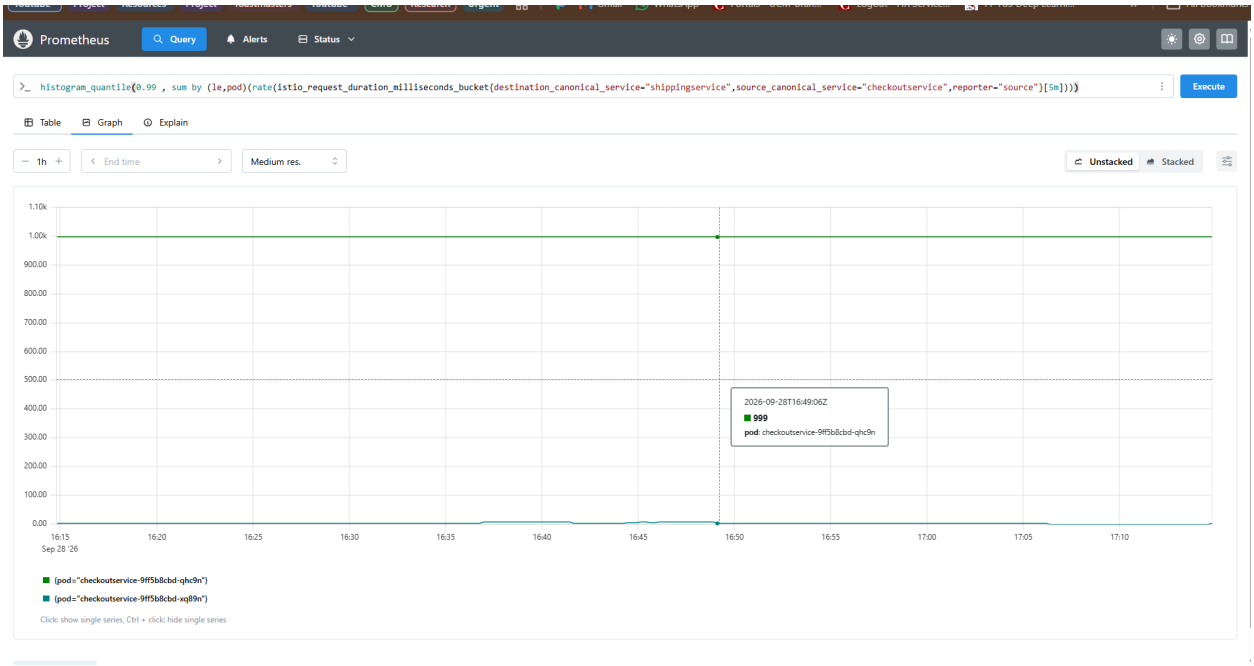


Figure 6: Task 3 – Prometheus graph view of the p99 Shipping service response time per `checkoutservice` pod: the delayed pod sits at about 1000 ms, the other near 0.

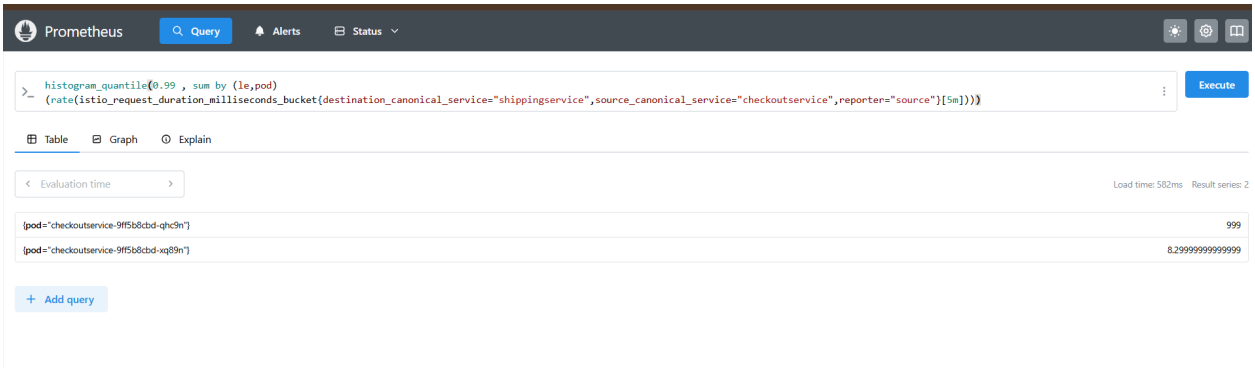


Figure 7: Task 3 – Prometheus table view per `checkoutservice` pod: 999 ms for the labelled pod versus about 8.3 ms for the unlabelled pod.

5. Task 4 – Showing Results in Grafana

5.1. Query and panel configuration

Grafana was launched with `istioctl dashboard grafana`, and Prometheus was selected as the data source in the Explore tab. The body of the histogram query was entered in *Code* mode, with the `histogram_quantile(0.99, ...)` wrapper and any `[n:m]` subrange removed, so that Grafana receives the raw bucket rates:

```
sum by (le)
  (rate(istio_request_duration_milliseconds_bucket{
     destination_canonical_service="shippingservice",
     source_canonical_service=~"checkoutservice|frontend",
     reporter="source"}[5m]))
```

Listing 8: Task 4: PromQL query body used in the Grafana panel

The query was added to a new dashboard, and the panel was configured as follows to obtain the histogram appearance: Visualization set to **Bar gauge**, query Format set to **Heatmap** (which converts the cumulative `le` buckets into per-bucket counts), and Type set to **Instant** so that only the most recent bucket values are displayed.

5.2. Results

Figure 8 shows the resulting panel. It has two clear clusters: a large count in the low-millisecond buckets (peaking at the 1 ms bucket), which corresponds to the undelayed calls, and a second mass at the high end (the 1000 ms and `+Inf` buckets), which corresponds to calls affected by the 1 s injected delay. Figure 9 shows the edit screen with the query, visualization type and data options.



Figure 8: Task 4 – Grafana Bar gauge panel showing the Shipping service response-time histogram buckets (ms).

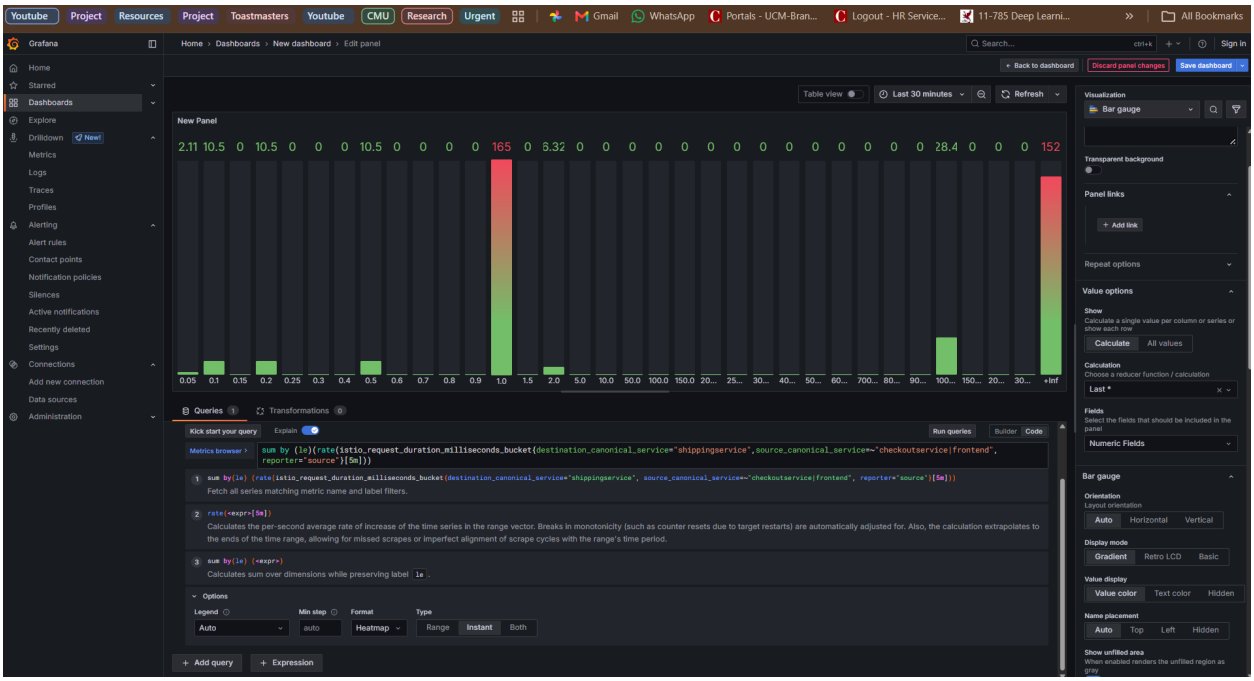


Figure 9: Task 4 – Grafana panel edit screen: PromQL query in Code mode, Visualization = Bar gauge, Format = Heatmap, Type = Instant.

6. Extra Credit Task 1 – Heatmap Visualization

The same bucket data can be viewed as a Grafana **Heatmap**, which is effectively the histogram rotated by 90 degrees and repeated over time. The Task 3 setup (two checkout pods, one delayed) was used with the query below; the Visualization was set to Heatmap, Options→Format to Heatmap, and Options→Type to Range.

```
sum by (le)
  (rate(istio_request_duration_milliseconds_bucket{
     destination_canonical_service="shippingservice",
     source_canonical_service="checkoutservice",
     reporter="source"}[5m]))
```

Listing 9: Extra Credit Task 1: PromQL query for the heatmap (calls from `checkoutservice`)

With the pod-selective delay applied (Figure 10), the heatmap shows two separate horizontal bands: a low band below about 5 ms from the unlabelled pod, and a band at roughly 1000 ms from the delayed pod. After the Istio rule was deleted (Figure 11), the 1000 ms band disappears and only the low-latency band remains. This is the expected switch between two bands and a single band.

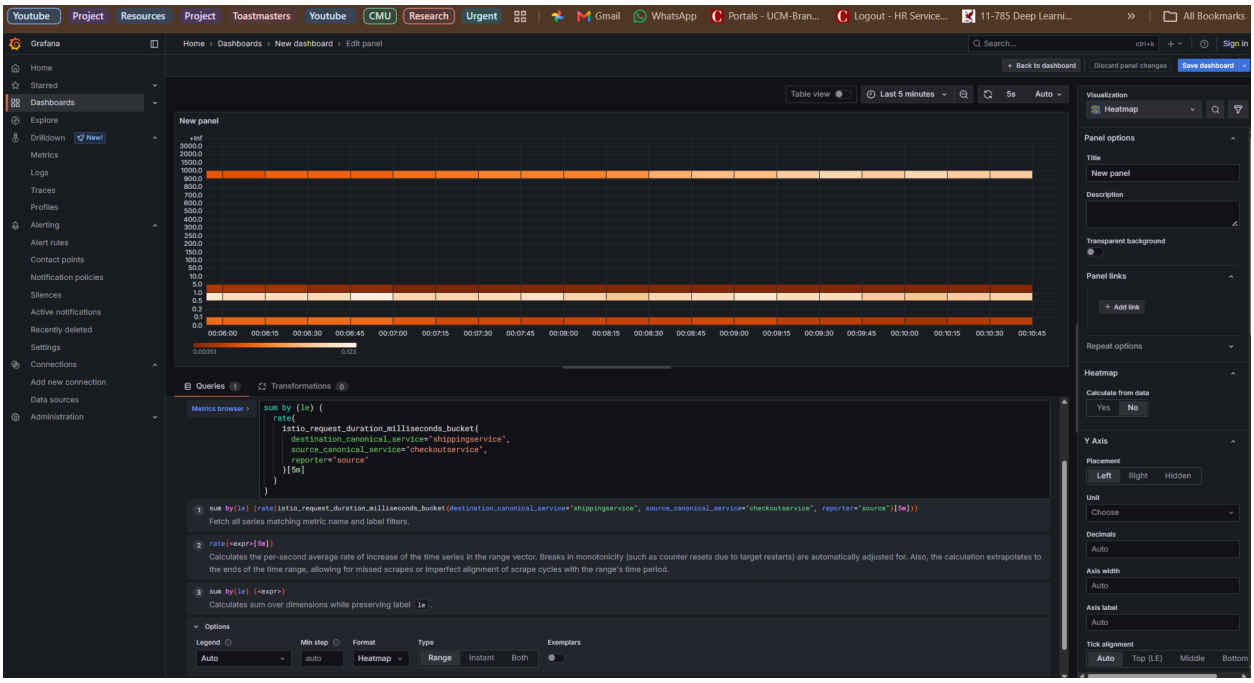


Figure 10: Extra Credit Task 1 – Grafana heatmap with the selective 1 s delay applied: a low-latency band and a second band at about 1000 ms.



Figure 11: Extra Credit Task 1 – Grafana heatmap after deleting the Istio delay rule: only the low-latency band remains.

7. Summary

Table 1 summarizes the p99 response times observed for the Shipping service across the tasks.

Table 1: Summary of observed p99 Shipping service response times (source-reported).

Task	Series	Observed p99
Task 1 (healthy)	`checkoutservice` `frontend`	≈ 3–4.7 ms 2 ms
Task 2 (checkout delay)	`checkoutservice` `frontend`	≈ 995–998 ms 2 ms
Task 3 (one-pod delay)	pod `qhc9n` (`fault=delay`) pod `xq89n`	999 ms ≈ 8.3 ms

Separating the metrics with `sum by()` on the right label (`source_canonical_service` for call paths, `pod` for replicas) makes it possible to isolate a fault that aggregate statistics would hide or blur. Reading the metrics from the source side is essential, because the injected delay is added in the client-side proxy and never appears in the destination's own timings. Grafana's Bar gauge and Heatmap views present the same bucket data as a snapshot distribution and as a distribution over time.